

404 CTF - Édition 2026

Thème : Les grandes figures scientifiques françaises !

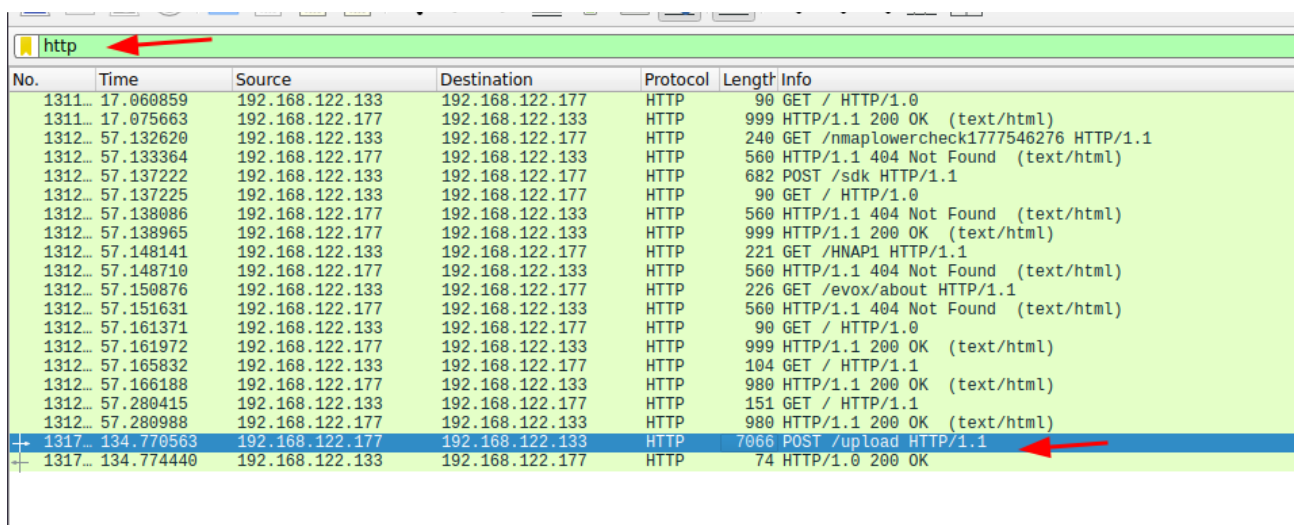
Writeup : Exfiltration Kantik [3/3]

Catégorie : Analyse Forensique

Auteur : SayWess

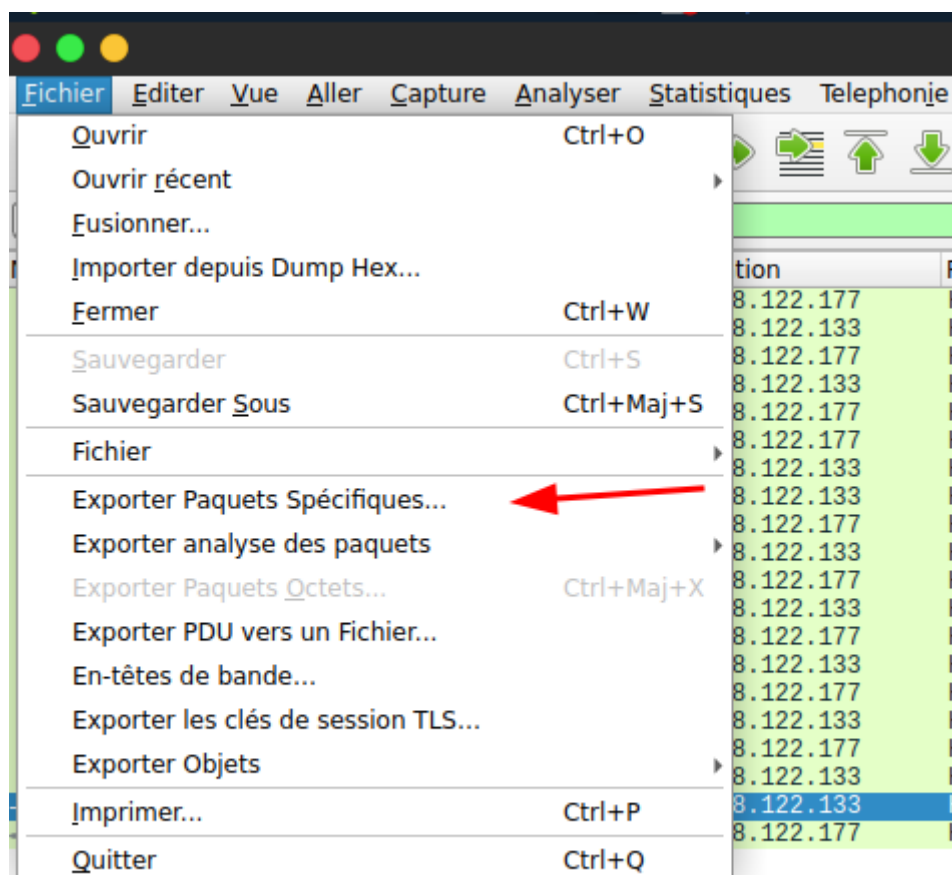
1. Énoncé

2. Résolution



No.	Time	Source	Destination	Protocol	Length	Info
1311...	17.060859	192.168.122.133	192.168.122.177	HTTP	90	GET / HTTP/1.0
1311...	17.075663	192.168.122.177	192.168.122.133	HTTP	999	HTTP/1.1 200 OK (text/html)
1312...	57.132620	192.168.122.133	192.168.122.177	HTTP	240	GET /nmaplowercheck177546276 HTTP/1.1
1312...	57.133364	192.168.122.177	192.168.122.133	HTTP	560	HTTP/1.1 404 Not Found (text/html)
1312...	57.137222	192.168.122.133	192.168.122.177	HTTP	682	POST /sdk HTTP/1.1
1312...	57.137225	192.168.122.133	192.168.122.177	HTTP	90	GET / HTTP/1.0
1312...	57.138086	192.168.122.177	192.168.122.133	HTTP	560	HTTP/1.1 404 Not Found (text/html)
1312...	57.138965	192.168.122.177	192.168.122.133	HTTP	999	HTTP/1.1 200 OK (text/html)
1312...	57.148141	192.168.122.133	192.168.122.177	HTTP	221	GET /HNAP1 HTTP/1.1
1312...	57.148710	192.168.122.177	192.168.122.133	HTTP	560	HTTP/1.1 404 Not Found (text/html)
1312...	57.150876	192.168.122.133	192.168.122.177	HTTP	226	GET /evox/about HTTP/1.1
1312...	57.151631	192.168.122.177	192.168.122.133	HTTP	560	HTTP/1.1 404 Not Found (text/html)
1312...	57.161371	192.168.122.133	192.168.122.177	HTTP	90	GET / HTTP/1.0
1312...	57.161972	192.168.122.177	192.168.122.133	HTTP	999	HTTP/1.1 200 OK (text/html)
1312...	57.165832	192.168.122.133	192.168.122.177	HTTP	104	GET / HTTP/1.1
1312...	57.166188	192.168.122.177	192.168.122.133	HTTP	980	HTTP/1.1 200 OK (text/html)
1312...	57.280415	192.168.122.133	192.168.122.177	HTTP	151	GET / HTTP/1.1
1312...	57.280988	192.168.122.177	192.168.122.133	HTTP	980	HTTP/1.1 200 OK (text/html)
1317...	134.770563	192.168.122.177	192.168.122.133	HTTP	7066	POST /upload HTTP/1.1
1317...	134.774440	192.168.122.133	192.168.122.177	HTTP	74	HTTP/1.0 200 OK

Donc je souhaite extraire les données de cette requête via le chemin “Exporter Paquets Octets” sur la trame “Data” :



Suite à cette exportation, je souhaite savoir de quel type de fichier il s'agit avec la commande file, et regarder rapidement à l'intérieur avec la commande strings :

```
[ramisto@parrot]-[~/Documents/404-ctf-2026/exfiltration-kantik]
$ file export
export: openssl enc'd data with salted password
```

```
[ramisto@parrot]-[~/Documents/404-ctf-2026/exfiltration-kantik]
$ strings export
Salted__
FcURz
jKtG
@V}glv
0sHvX!H
nxTe
[PvA
!UW]
RJ,9
```

Il s'agit donc d'un fichier chiffré avec openssl avec un salt en complément (d'après wikipedia, "le sel est une donnée aléatoire ajoutée en entrée à une fonction à sens unique qui hache des données, un mot de passe ou une phrase de passe").

Et effectivement, j'ai essayé de déchiffrer le fichier mais il demande une passphrase

```
$ mv export encrypted_export
```

```
$ openssl enc -d -aes-256-cbc -md sha256 -in encrypted_export -out decrypted-export
```

```
[ramisto@parrot]-[~/Documents/404-ctf-2026/exfiltration-kantik]
$ openssl enc -d -aes-256-cbc -md sha256 -in encrypted_export -out decrypted-export
enter AES-256-CBC decryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
bad decrypt
80A713682B7F0000:error:1C800064:Provider routines:ossl_cipher_unpadblock:bad decrypt:../p
ntations/ciphers/ciphercommon_block.c:107:
[x]-[ramisto@parrot]-[~/Documents/404-ctf-2026/exfiltration-kantik]
$
```

Dans la description, il est recommandé d'utiliser également le dump de la mémoire volatile du serveur victime des précédents challenges exfiltration kantik.

J'ai donc cherché des traces de commandes openssl dans ce dump mémoire :

```
$ strings memory_dump.lime | grep "openssl enc"
```

```
[ramisto@parrot]~/Documents/404-ctf-2026/exfiltration-kantik]
$ strings memory_dump.lime | grep "openssl enc"
openssl enc -aes-256-cbc -salt -in /tmp/.backup/data.tar.gz -out /tmp/.encrypted_data.enc -k $ENCRYPT_KEY
openssl enc -aes-256-cbc -salt -in /tmp/.backup/data.tar.gz -out /tmp/.encrypted_data.enc -k $ENCRYPT_KEY
openssl enc -aes-256-cbc -salt -in /tmp/.backup/data.tar.gz -out /tmp/.encrypted_data.enc -k $ENCRYPT_KEY
openssl enc -aes-256-cbc -salt -in /tmp/.backup/data.tar.gz -out /tmp/.encrypted_data.enc -k $ENCRYPT_KEY
openssl enc -aes-256-cbc -salt -in /tmp/.backup/data.tar.gz -out /tmp/.encrypted_data.enc -k $ENCRYPT_KEY
openssl enc -aes-256-cbc -salt -in /tmp/.backup/data.tar.gz -out /tmp/.encrypted_data.enc -k $ENCRYPT_KEY
[ramisto@parrot]~/Documents/404-ctf-2026/exfiltration-kantik]
$
```

Nous avons effectivement une commande openssl qui chiffre une archive de backup, et ceci avec une clé stockée dans la variable d'environnement \$ENCRYPT_KEY.

Donc allons voir les traces des commandes qui utilisent cette variable !

```
$ strings memory_dump.lime | grep "ENCRYPT_KEY"
```

```
[x]~[ramisto@parrot]~/Documents/404-ctf-2026/exfiltration-kantik]
$ strings memory_dump.lime | grep "ENCRYPT_KEY"
export ENCRYPT_KEY=$(printf '%x' $(date +%s))
echo $ENCRYPT_KEY
openssl enc -aes-256-cbc -salt -in /tmp/.backup/data.tar.gz -out /tmp/.encrypted_data.enc -k $ENCRYPT_KEY
openssl enc -aes-256-cbc -salt -in /tmp/.backup/data.tar.gz -out /tmp/.encrypted_data.enc -k $ENCRYPT_KEY
export ENCRYPT_KEY=$(printf '%x' $(date +%s))
export ENCRYPT_KEY=$(printf '%x' $(date +%s))
echo $ENCRYPT_KEY
openssl enc -aes-256-cbc -salt -in /tmp/.backup/data.tar.gz -out /tmp/.encrypted_data.enc -k $ENCRYPT_KEY
echo $ENCRYPT_KEY
echo $ENCRYPT_KEY
echo $ENCRYPT_KEY
openssl enc -aes-256-cbc -salt -in /tmp/.backup/data.tar.gz -out /tmp/.encrypted_data.enc -k $ENCRYPT_KEY
export ENCRYPT_KEY=$(printf '%x' $(date +%s))
export ENCRYPT_KEY=$(printf '%x' $(date +%s))
echo $ENCRYPT_KEY
openssl enc -aes-256-cbc -salt -in /tmp/.backup/data.tar.gz -out /tmp/.encrypted_data.enc -k $ENCRYPT_KEY
echo $ENCRYPT_KEY
export ENCRYPT_KEY=$(printf '%x' $(date +%s))
openssl enc -aes-256-cbc -salt -in /tmp/.backup/data.tar.gz -out /tmp/.encrypted_data.enc -k $ENCRYPT_KEY
echo $ENCRYPT_KEY
[ramisto@parrot]~/Documents/404-ctf-2026/exfiltration-kantik]
$
```

Donc finalement, le contenu de cette variable (et donc de notre passphrase openssl) est un timestamp converti en hexadécimal.

Après quelques recherches sur le net, j'ai eu ces infos :

- date +%s → timestamp Unix (secondes depuis epoch)
- printf '%x' → conversion en hexadécimal

La partie epoch désigne le nombre de secondes écoulées depuis le 1er janvier 1970 à 00:00:00 UTC

Nous savons qu'il faut obtenir un timestamp.

Je suis retourné voir la trace de timestamp dans les échanges HTTP (au moment de l'exfiltration des documents) :

1312...	57.165832	192.168.122.133	192.168.122.177	HTTP	104 GET / HTTP/1.1
1312...	57.166188	192.168.122.177	192.168.122.133	HTTP	980 HTTP/1.1 200 OK (text/html)
1312...	57.280415	192.168.122.133	192.168.122.177	HTTP	151 GET / HTTP/1.1
1312...	57.280988	192.168.122.177	192.168.122.133	HTTP	980 HTTP/1.1 200 OK (text/html)
1317...	134.770563	192.168.122.177	192.168.122.133	HTTP	7066 POST /upload HTTP/1.1
1317...	134.774440	192.168.122.133	192.168.122.177	HTTP	74 HTTP/1.0 200 OK

```
▼ Frame 131728: 7066 bytes on wire (56528 bits), 7066 bytes captured (56528 bits)
  Encapsulation type: Linux cooked-mode capture v2 (210)
  Arrival Time: Apr 30, 2026 12:52:34.016065000 CEST
  UTC Arrival Time: Apr 30, 2026 10:52:34.016065000 UTC
  Epoch Arrival Time: 1777546354.016065000
  [Time shift for this packet: 0.000000000 seconds]
  [Time delta from previous captured frame: 0.000019000 seconds]
  [Time delta from previous displayed frame: 77.489575000 seconds]
  [Time since reference or first frame: 134.770563000 seconds]
  Frame Number: 131728
  Frame Length: 7066 bytes (56528 bits)
```

Et je me suis également souvenu que l'attaquant a utilisé telnet pour obtenir un accès sur le serveur victime, et j'ai pris le dernier paquet telnet de l'attaquant :

1313...	95.421358	192.168.122.177	192.168.122.133	TELNET	95 23 bytes data
1313...	95.421841	192.168.122.133	192.168.122.177	TELNET	109 37 bytes data
1313...	95.421856	192.168.122.177	192.168.122.133	TELNET	112 40 bytes data
1314...	95.463463	192.168.122.177	192.168.122.133	TELNET	1057 985 bytes data
1314...	95.464581	192.168.122.133	192.168.122.177	TELNET	77 5 bytes data
1314...	95.464919	192.168.122.177	192.168.122.133	TELNET	88 16 bytes data
1314...	95.507549	192.168.122.177	192.168.122.133	TELNET	80 8 bytes data

```
▼ Frame 131408: 80 bytes on wire (640 bits), 80 bytes captured (640 bits)
  Encapsulation type: Linux cooked-mode capture v2 (210)
  Arrival Time: Apr 30, 2026 12:51:54.753051000 CEST
  UTC Arrival Time: Apr 30, 2026 10:51:54.753051000 UTC
  Epoch Arrival Time: 1777546314.753051000
  [Time shift for this packet: 0.000000000 seconds]
  [Time delta from previous captured frame: 0.000068000 seconds]
  [Time delta from previous displayed frame: 0.042630000 seconds]
  [Time since reference or first frame: 95.507549000 seconds]
  Frame Number: 131408
  Frame Length: 80 bytes (640 bits)
  Capture Length: 80 bytes (640 bits)
```

Donc finalement, le bon timestamp pourrait se trouver durant la période où :

- l'attaquant effectue une exfiltration HTTP
- il chiffre les données obtenues avec openssl
- et il coupe sa session telnet

Je ne peux pas deviner le timestamp durant cette période, donc j'ai créé un script bash qui permet de faire du bruteforce openssl avec une wordlist de plusieurs timestamps convertis en hexadécimal (avec l'aide de claude.ai, car je suis nul pour construire des scripts, et je suis un fainéant).

Et ensuite, si le script bash réussit, il m'affiche le timestamp en clair et en hexadécimal, puis il liste les données extraites :

```
GNU nano 8.4
1 for ts in $(seq 1777546314 1777546354); do
2   key=$(printf '%x' $ts)
3   openssl enc -d -aes-256-cbc -salt \
4     -in encrypted_export \
5     -out test_out.bin \
6     -k $key 2>/dev/null
7   if tar tzf test_out.bin 2>/dev/null | grep -q "."; then
8     echo "Clé trouvée : ts=$ts key=$key"
9     tar tzf test_out.bin
10    break
11  fi
12 done
13
```

Tout fonctionne ! le bon timestamp est 1777546341 et si on fait print '%x' dessus cela nous donne 69f33465.

```
(venv) | [francis@paul100] [ /documents/404-ctf-2026/extraction-keys ]
└─ $sh timestamp.sh
Clé trouvée : ts=1777546341 key=69f33465
2026_04_18-Rapport_Confidentiel.pdf
quantum_lab/
quantum_lab/projets/
quantum_lab/projets/quantum_crypto/
quantum_lab/projets/quantum_simulation/
quantum_lab/projets/qbit_entanglement/
quantum_lab/donnees/
quantum_lab/donnees/simulations/
quantum_lab/donnees/simulations/ai_quantique_2026_02_05.log
quantum_lab/donnees/simulations/simulation_multiverse_2026_03_04.log
quantum_lab/donnees/analysis/
quantum_lab/donnees/analysis/synced_experimental_data/
quantum_lab/donnees/analysis/synced_experimental_data/incident_2026_02
quantum_lab/donnees/analysis/synced_experimental_data/experience_2026
```

Et à ce moment là, nous pouvons déchiffrer les données avec la bonne passphrase, et je découvre un rapport “confidentiel” et le dossier “quantum_lab” :

```
$ openssl enc -d -aes-256-cbc -md sha256 -in encrypted_export -out data.tar.gz -k 69f33465
```

```
$ tar xzf data.tar.gz
```

```
[ramisto@parrot]-[~/Documents/404-ctf-2026/exfiltration-kantik]
└─$ openssl enc -d -aes-256-cbc -salt \
    -in encrypted_export \
    -out data.tar.gz \
    -k 69f33465
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
```

```
Using -iter or -pbkdf2 would be better.
[ramisto@parrot]-[~/Documents/404-ctf-2026/exfiltration-kantik]
└─$ tar xzf data.tar.gz
```

```
[ramisto@parrot]-[~/Documents/404-ctf-2026/exfiltration-kantik]
└─$ ls
2026_04_18-Rapport_Confidentiel.pdf  data.tar.gz      memory_dump.lime  quantum_lab
all_strings.txt                     encrypted_export network_capture.pcap uncrpyted_export
[ramisto@parrot]-[~/Documents/404-ctf-2026/exfiltration-kantik]
└─$
```

Voici le contenu du fichier *PDF 2026_04_18-Rapport_Confidentiel.pdf* :

RAPPORT CONFIDENTIEL

2026-04-28

PROGRAMME NQF-2026

INTERNE UNIQUEMENT

Author: Mr. Cap
Laboratoire: Institut National de Calcul Kantik
Site Principale: Plateau de Saclou
Classification: Secret Recherche Niveau STRATOSPHERIC
Date: 2026-04-28
Reference ID: [REF-0126-2026]

Ici le chef de l'équipe Ciel-Azure au rapport. Notre équipe de recherche a atteint des résultats significatifs dans le développement de qubits supraconducteurs à haute fidélité pour nos ordinateurs quantiques. Les résultats préliminaires indiquent une amélioration substantielle de la cohérence et une réduction des taux d'erreur, positionnant notre programme à la pointe de la recherche quantique mondiale. Ces résultats ne doivent pas tomber entre de mauvaises mains, il s'agit d'informations pouvant révolutionner les ordinateurs quantiques à tout jamais, et en conséquence notre monde tout entier !!!!

Je vous informe de la nouvelle méthode que nous avons découverte pour atteindre un tel niveau de performance, je compte sur vous pour garder cela strictement confidentiel.

La plus grosse découverte que nous ayons faite est qu'il faut.....mettre des petits drapeaux tricolores sur les qubits, et cela a permis d'atteindre une fidélité de 99.99% sur 2048 qubits supraconducteurs, ce qui est une performance inédite dans le domaine. Je ne connais pas encore la raison physique exacte de ce phénomène, mais je soupçonne que cela pourrait être lié à une meilleure isolation des qubits contre les perturbations environnementales et les tempêtes interstellaires que notre planète affronte, ou peut-être une modification subtile de la dynamique quantukénienne... ou les qbits sont tout simplement très fans de la France... peut-être ont-ils même leur propre conscience ??? Ce ne sont encore que des hypothèses, mais nous allons continuer à investiguer pour comprendre les mécanismes sous-jacents à cette amélioration spectaculaire.

En attendant, voici les données brutes de nos expériences, que je vous demande de garder strictement confidentielles et de ne pas partager avec quiconque en dehors de notre équipe de recherche. Ces données sont quantiques et aiment sortir du cadre, alors faites attention à ne pas les laisser s'échapper ! 3430344354467b

Je vous tiendrai informé de nos prochaines avancées.

Signé: Chef d'équipe Ciel-Azure,
"Liberté, Égalité, Superposition"

P.S.: Les qubits m'ont demandé une augmentation. Je ne sais pas quoi leur répondre.

=====

FIN DU RAPPORT - À DÉTRUIRE APRÈS LECTURE
(mais gardez les drapeaux)

=====

Ce document contient des informations scientifiques sensibles.
Toute diffusion non autorisée est strictement interdite.

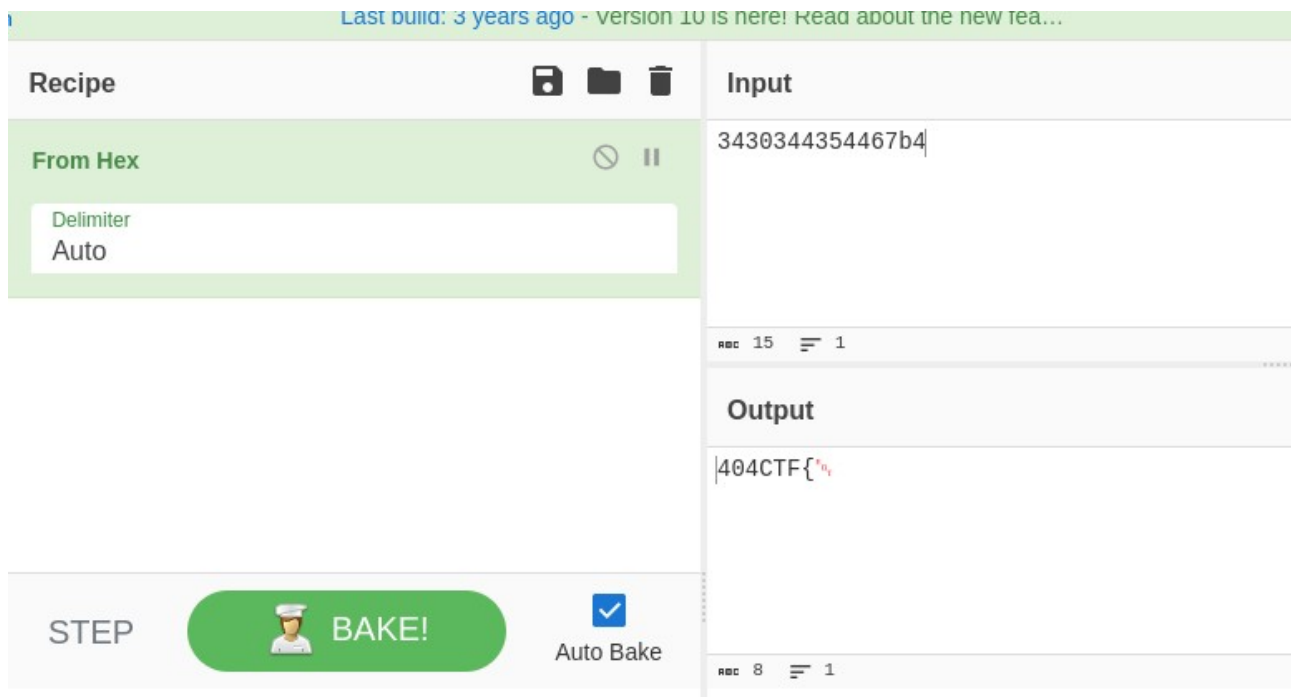
Page 1

Je vois qu'il y a un élément bizarre en bas du document :

Je vous demande de garder strictement
ces données de notre équipe de recherche. Ces
données sont quantiques et aiment sortir du cadre, alors faites attention à ne pas les laisser s'échapper ! 3430344354467b



J'ai donné cet élément à CyberChef, et c'est encore de l'hexadécimal (décidement!) :



De plus, cela ressemble à un début de flag, mais tout est coupé.

Je pleure :-/



Je demande donc un coup de main à mon poto claude.ai, et il m'indique que le texte déborde hors de la marge, et nous pouvons le lire au-delà de cette marge avec la lib "PyMuPDF" :

Le PDF contient le flag encodé en hexadécimal, mais le texte déborde hors de la marge visible de la page. Il faut lire au-delà des limites avec PyMuPDF :

```
python

import fitz, re

doc = fitz.open('2026_04_18-Rapport_Confidentiel.pdf')
page = doc[0]

# Lire avec un rectangle plus grand que la page (595x841pt)
full_rect = fitz.Rect(-100, -100, 1000, 1000)
text = page.get_text("text", clip=full_rect)

# Chercher la chaîne hex
idx = text.find('3430')
print(text[idx:idx+120])
# → 3430344354467b495f53573334525f...
```

Le script agrandi le "rectangle" par rapport à la page, et m'affiche la chaîne hexadécimal en sortie.

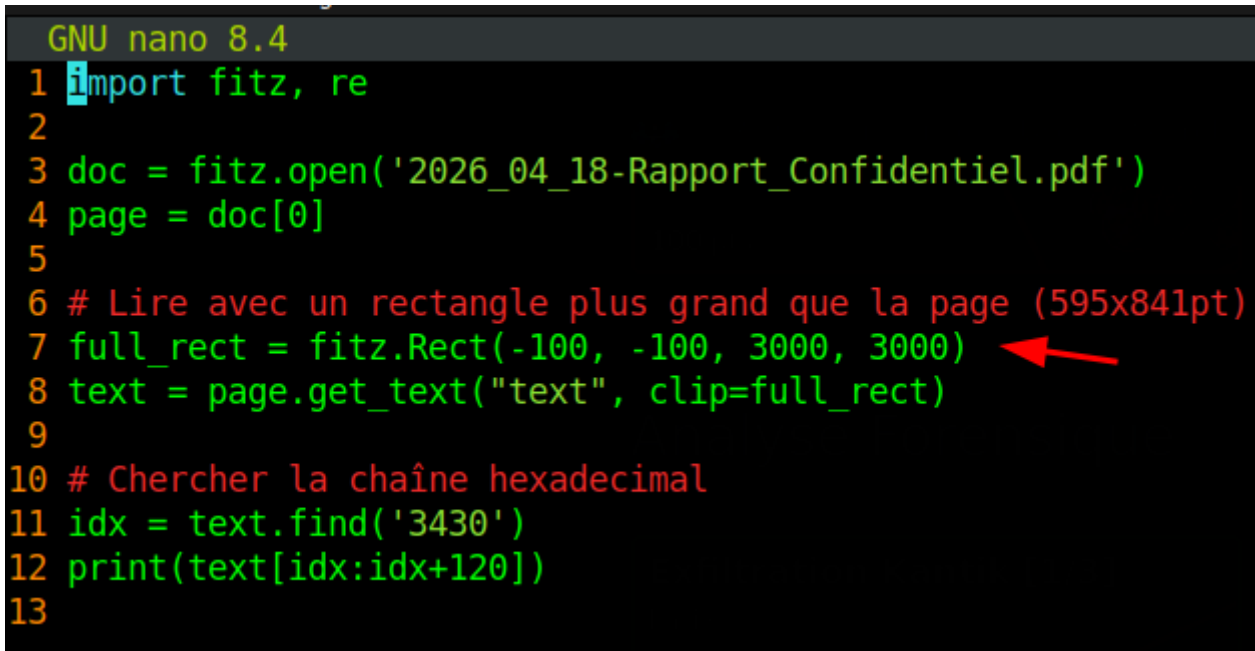
Donc je me place dans un environnement virtuel python, j'installe la lib "PyMuPDF", et je test le script.

```
➡ $pip install PyMuPDF
Collecting PyMuPDF
  Downloading pymupdf-1.27.2.3-cp310-abi3-manylinux_2_28_x86_64.whl.metadata (24 kB)
  Downloading pymupdf-1.27.2.3-cp310-abi3-manylinux_2_28_x86_64.whl (25.0 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 25.0/25.0 MB 54.1 MB/s eta 0:00:00
Installing collected packages: PyMuPDF
Successfully installed PyMuPDF-1.27.2.3
```

```
(venv) └─[ramisto@parrot]─[~/Documents/404-ctf-2026/exfiltration-kantik]
➡ $python3 test.py
3430344354467b495f53573334525f544831535f31535f4e30545f345f4a304b335f31545f5233344c4c595f5730524b3
Je vous tiendrai infor
```

J'ai dû effectuer plusieurs essais et modifier cette ligne dans le script comme ci-dessous pour avoir la chaîne complète :

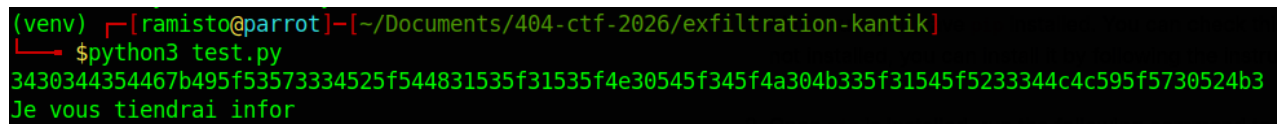
```
full_rect = fitz.Rect(-100, -100, 3000, 3000)
```



```
GNU nano 8.4
1 import fitz, re
2
3 doc = fitz.open('2026_04_18-Rapport_Confidentiel.pdf')
4 page = doc[0]
5
6 # Lire avec un rectangle plus grand que la page (595x841pt)
7 full_rect = fitz.Rect(-100, -100, 3000, 3000)
8 text = page.get_text("text", clip=full_rect)
9
10 # Chercher la chaîne hexadecimal
11 idx = text.find('3430')
12 print(text[idx:idx+120])
13
```

Ensuite, j'obtiens la chaîne hexadécimale en entière :

```
3430344354467b495f53573334525f544831535f31535f4e30545f345f4a304b335f31545f5233344c4
c595f5730524b3
```



```
(venv) [ramisto@parrot]~/Documents/404-ctf-2026/exfiltration-kantik
└─$ python3 test.py
3430344354467b495f53573334525f544831535f31535f4e30545f345f4a304b335f31545f5233344c4c595f5730524b3
Je vous tiendrai infor
```

Et enfin, je donne cette chaîne à CyberChef pour m'afficher le message via une conversion hex -> ASCII.

4. Obtention du Flag

Une fois toutes ces informations rassemblées, cela donne :

404CTF{I_SW34R_TH1S_1S_N0T_4_J0K3_1T_R34LLY_W0RK5_B3L1EV3_M3}